TRANSPORT AND TELECOMMUNICATION INSTITUTE

ENGINEERING FACULTY

Practical work N2

Course: **Computer Systems Structure**

Theme: **RISC Processor: AVR ASM**

Student: Igors Oļeiņikovs
Student code: 93642
Group: 4501BTA

Riga
2026

CONTENTS

Contents

Module Code	**UFCFCW-24-0**
Module Title	**Computer System Structure part 1**
Lecturer	**Vasil Peravoshchykau**
Students Name:	**Igors Oļeiņikovs**
Year	**2025/2026**
Semester	**II Semester**
Assessment Topic	**RISC Processor: AVR ASM**

Assignment	Delivered/Not Delivered	Grade
Exercise 1		/20
Exercise 2		/20
Exercise 3		/20
Exercise 4		/20
	Total:	**/100**

1. EXERCISE 1

<u>Exercise 1:</u> *Setting up the device connection*

Objective: Create the test setup, adapt the starter program to the required external LED connection, and verify the blinking behavior.

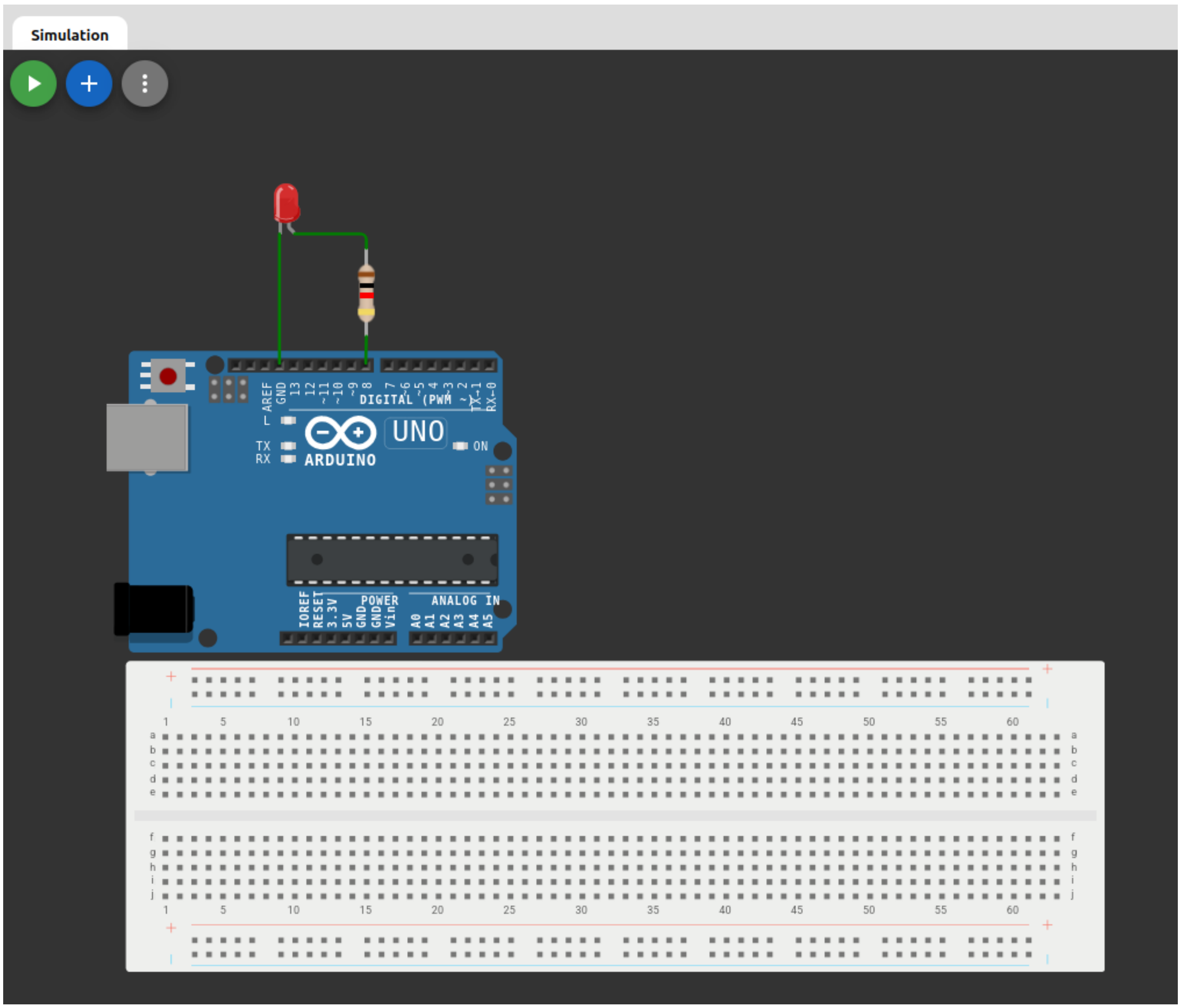

Fig. 1 Initial wokwi test setup interface

Implementation Summary

The starter Wokwi project from the assignment blinks the built-in LED on Arduino Uno pin 13. The task itself, however, requires an external LED connected to pin **8** through a 1kOhm resistor. To match the requirement, original assembly example was adapted from PB5 to PB0, because on the ATmega328P microcontroller PB0 corresponds to Arduino digital pin 8.

Program Logic

- Configure PB0 as an output by setting bit 0 in DDRB.
- Toggle the LED state by writing to PINB bit 0.
- Call a reusable millisecond delay routine to create a one-second period between toggles
- Repeat the sequence indefinitely

Challenge and Strategy

The main challenge was the mismatch between the provided online example and the written hardware task. Instead of keeping the starter project unchanged, the solution was aligned with the assignment by moving the output signal from PB5/pin 13 to PB0/pin 8. This preserves the same timing logic while producing the correct physical wiring.

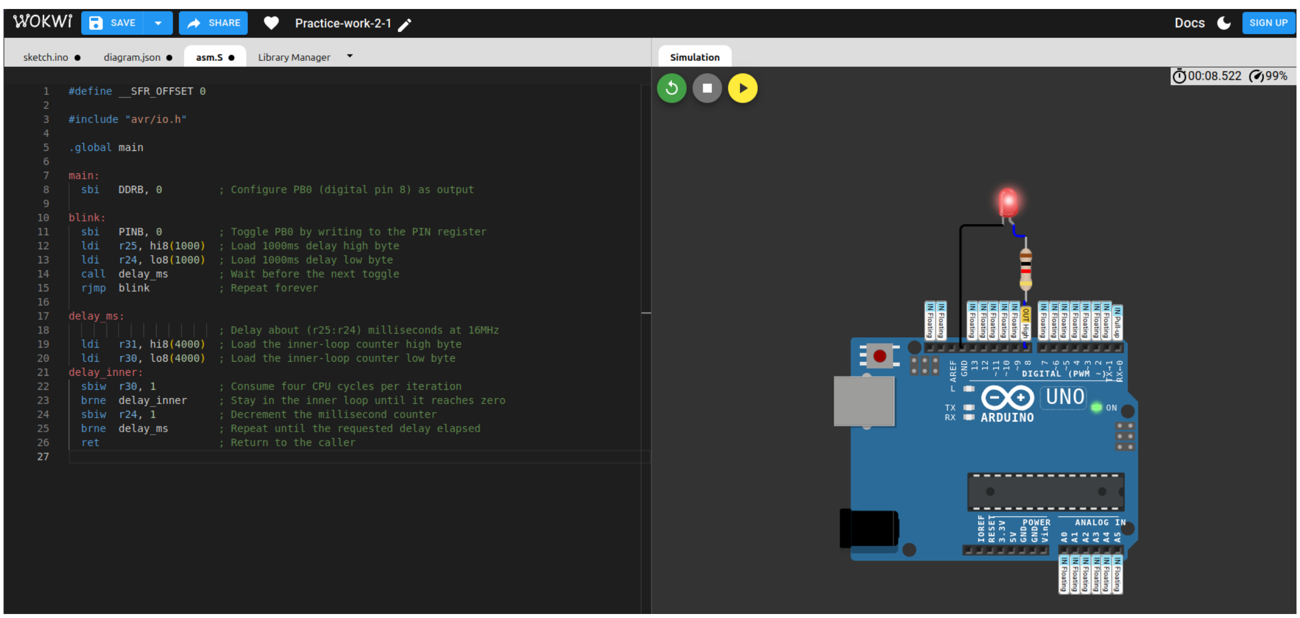

Fig. 2 Exercise 1 Light on

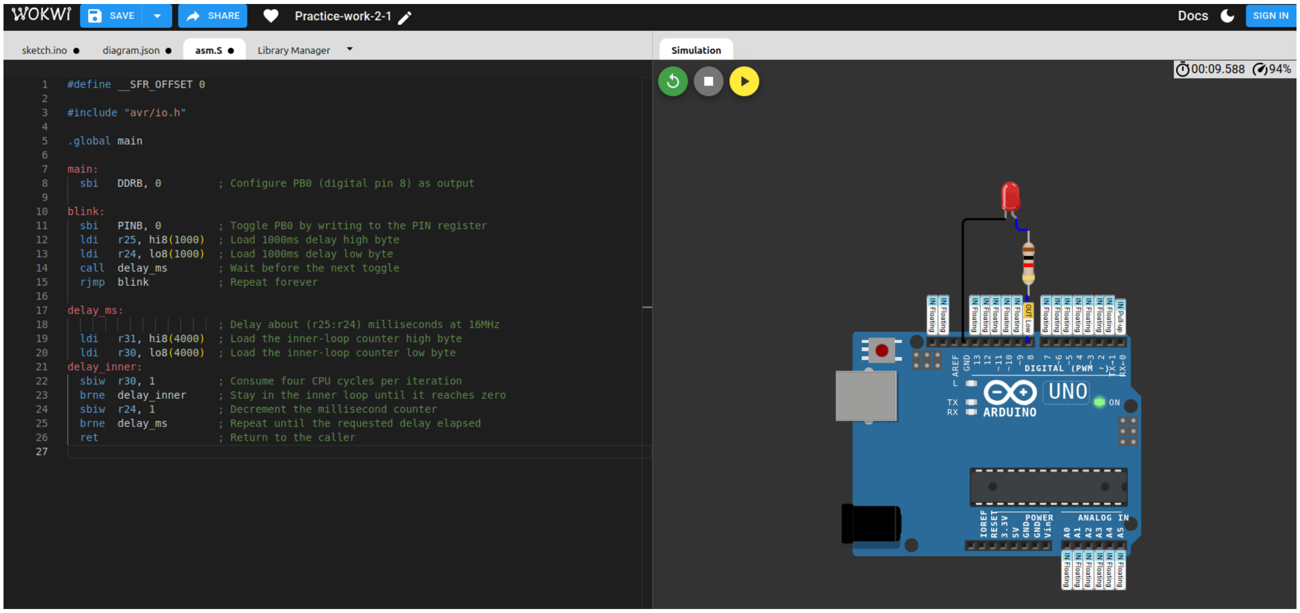

Fig. 3 Exercise 1 Light off

2. EXERCISE 2

<u>Exercise 2:</u> *Running lights on Arduino UNO board using AVR Assembler*

Objective: Connect five LEDs to pins 13 down to 9 and create a running lights effect with equal timing between all LED changes.

Implementation Summary

Five LEDs were connected to Arduino Uno pins 13, 12, 11, 10 and 9 through separate 1kOhm resistors. These pins correspond to PB5, PB4, PB3, PB2, and PB1 on the ATmega328P. The solution stores the active LED position in register r17. The program writes this bit pattern to PORTB, waits for the same delay interval, shifts the bit to the next LED, and resets the sequence after the last LED has been displayed.

Program Design and Logic

- PB5..PB1 are configured as outputs using DDRB = 0x3E.
- The current active LED is held in r17, initially set to 0x20 for PB5.
- After every output update, the same 1000ms delay is applied.
- The running light advances by executing lsr r17, which moves the active bit from PB5 toward PB1.
- When the shift reaches bit 0, the sequence restarts from PB5.

```
; Exercise 2
; Create running-lights on Arduino Uno pins 13..9 (PB5..PB1)

#define __SFR_OFFSET 0

#include "avr/io.h"

.global main

main:
  ldi   r16, 0x3E         ; Configure PB5..PB1 as LED outputs
  out   DDRB, r16         ; Write the output mask into DDRB
  ldi   r17, 0x20         ; Start with the LED on PB5 (digital pin 13)

run_lights:
  out   PORTB, r17        ; Show the current running-light position
  ldi   r25, hi8(1000)    ; Load a 1000ms pause high byte
  ldi   r24, lo8(1000)    ; Load a 1000ms pause low byte
  call  delay_ms          ; Keep the current LED on for the same time
  lsr   r17               ; Move the lit LED one position toward PB1
  cpi   r17, 0x01         ; Check whether the shift moved outside PB1..PB5
  brne  run_lights        ; Continue if a valid LED bit is still set
  ldi   r17, 0x20         ; Restart from PB5 after PB1 was displayed
  rjmp  run_lights        ; Repeat forever

delay_ms:
                          ; Delay about (r25:r24) milliseconds at 16MHz
  ldi   r31, hi8(4000)    ; Load the inner-loop counter high byte
  ldi   r30, lo8(4000)    ; Load the inner-loop counter low byte
delay_inner:
  sbiw  r30, 1            ; Spend four clock cycles per inner-loop pass
  brne  delay_inner       ; Stay here until the inner loop reaches zero
  sbiw  r24, 1            ; Decrement the millisecond counter
  brne  delay_ms          ; Repeat until the whole delay elapsed
  ret                     ; Return to the caller
```

Challenges and Strategies

The most important design requirement was equal timing between all LED transitions. To keep this property explicit, the code uses the same delay routine and the same delay constant before each shift operation. This makes the movement regular and easy to modify.

How the Time Pause Can Be Changed

The pause is controlled by the value loaded into registers r25:r24 immediately before call delay_ms. In the solution the value is 1000, which gives an approximately 1000ms interval. Increasing this value makes the light move more slowly, while decreasing it makes the effect faster.

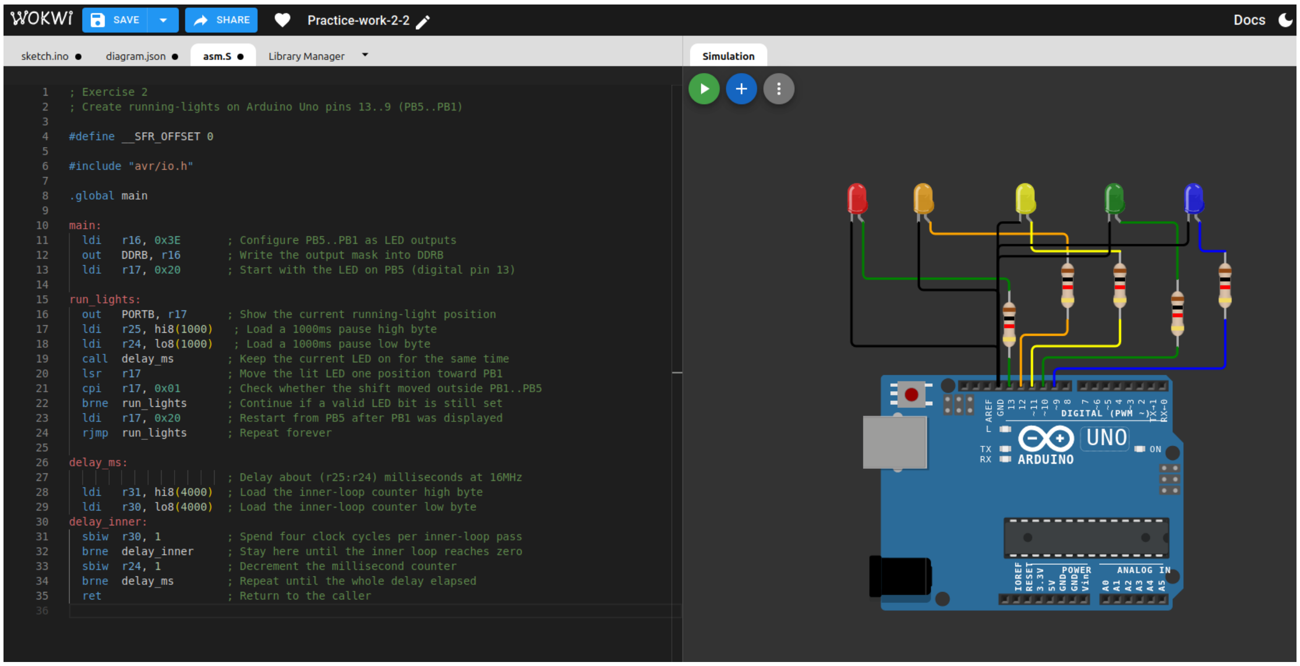

Fig. 4 Exercise 2

Video

Exercise-2.mp4

3. EXERCISE 3

Exercise 3: *LEDs and Buttons control on Arduino UNO using AVR Assembler*

Objective: Use one button on Port D to invert the odd/even state of five LEDs connected to Port B.

Implementation Summary

The same five LED outputs from Exercise 2 were reused. A pushbutton was added to PD2 (Arduino pin 2) and configured as an active-low input using the internal pull-up resistor. The initial output pattern enables odd LEDs only. Each confirmed button press toggles the five-bit LED state, so the display alternates between odd LEDs on and even LEDs on.

Program Design and Logic

- The LED outputs are still PB5..PB1, configured as outputs.
- The button is connected to PD2 and uses the internal pull-up, so the input reads HIGH when released and LOW when pressed.
- The initial pattern is 0x2A, which enables PB5, PB3, and PB1.
- The inversion mask is 0x3E, so eor r17, r18 swaps odd and even LED states in one instruction.
- A short 20ms debounce delay is inserted after press detection and after release.

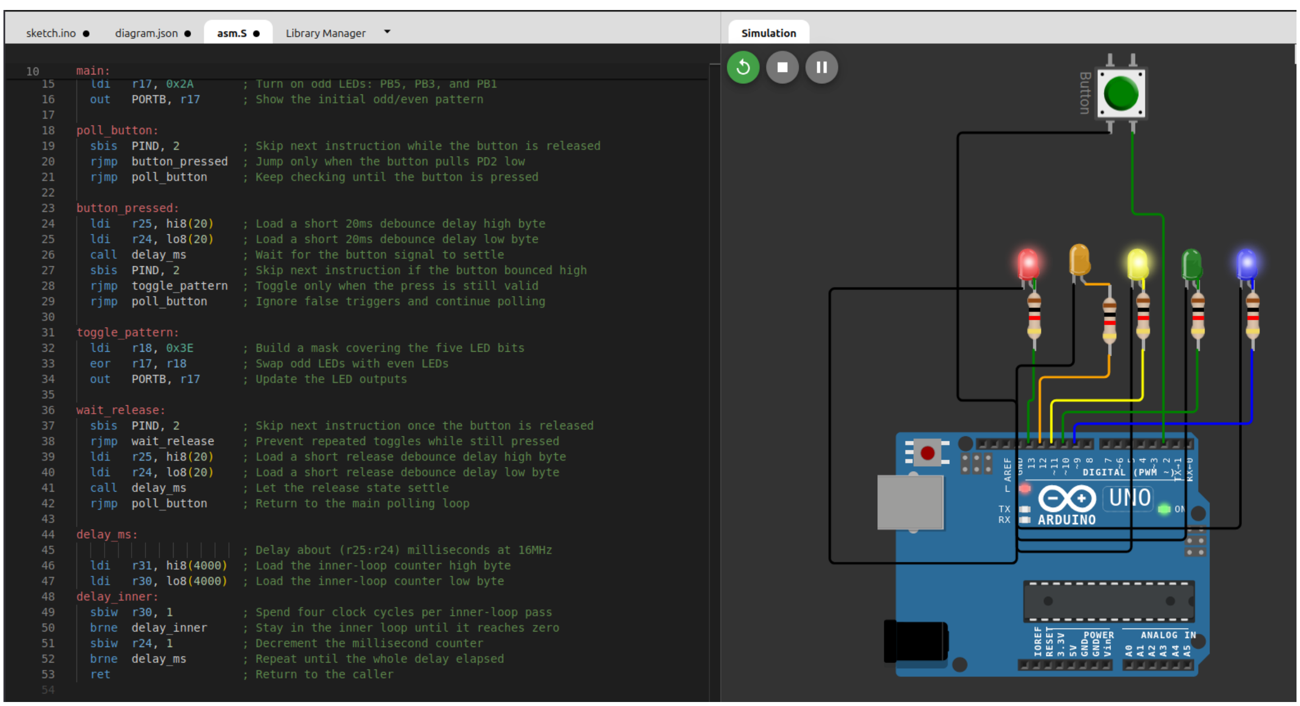

Fig. 5 Exercise 3

```
; Exercise 3
; Control five LEDs with one button

#define __SFR_OFFSET 0

#include "avr/io.h"

.global main

main:
  ldi   r16, 0x3E       ; Configure PB5..PB1 as LED outputs
  out   DDRB, r16       ; Enable the LED pins as outputs
  cbi   DDRD, 2         ; Keep PD2 (digital pin 2) as a button input
  sbi   PORTD, 2        ; Enable the internal pull-up on the button pin
  ldi   r17, 0x2A       ; Turn on odd LEDs: PB5, PB3, and PB1
  out   PORTB, r17      ; Show the initial odd/even pattern

poll_button:
  sbis  PIND, 2         ; Skip next instruction while the button is released
  rjmp  button_pressed  ; Jump only when the button pulls PD2 low
  rjmp  poll_button     ; Keep checking until the button is pressed

button_pressed:
  ldi   r25, hi8(20)    ; Load a short 20ms debounce delay high byte
  ldi   r24, lo8(20)    ; Load a short 20ms debounce delay low byte
  call  delay_ms        ; Wait for the button signal to settle
  sbis  PIND, 2         ; Skip next instruction if the button bounced high
  rjmp  toggle_pattern  ; Toggle only when the press is still valid
  rjmp  poll_button     ; Ignore false triggers and continue polling

toggle_pattern:
  ldi   r18, 0x3E       ; Build a mask covering the five LED bits
  eor   r17, r18        ; Swap odd LEDs with even LEDs
  out   PORTB, r17      ; Update the LED outputs

wait_release:
  sbis  PIND, 2         ; Skip next instruction once the button is released
  rjmp  wait_release    ; Prevent repeated toggles while still pressed
  ldi   r25, hi8(20)    ; Load a short release debounce delay high byte
  ldi   r24, lo8(20)    ; Load a short release debounce delay low byte
  call  delay_ms        ; Let the release state settle
  rjmp  poll_button     ; Return to the main polling loop

delay_ms:
                        ; Delay about (r25:r24) milliseconds at 16MHz
  ldi   r31, hi8(4000)  ; Load the inner-loop counter high byte
  ldi   r30, lo8(4000)  ; Load the inner-loop counter low byte
```

```
delay_inner:
  sbiw  r30, 1          ; Spend four clock cycles per inner-loop pass
  brne  delay_inner     ; Stay in the inner loop until it reaches zero
  sbiw  r24, 1          ; Decrement the millisecond counter
  brne  delay_ms        ; Repeat until the whole delay elapsed
  ret                   ; Return to the caller
```

Challenges and Strategies

The main risk in this exercise is repeated toggling caused by mechanical switch bounce or by holding the button down. The solution addresses this in two ways: it adds a short debounce delay after the button state changes, and it waits for the button to be released before it accepts another press. This keeps each press associated with a single LED inversion event.

Video

4. EXERCISE 4

<u>Exercise 4:</u> *7-segment display with Arduino UNO using AVR Assembler*

Objective: Display a countdown from 9 to 0, activate a buzzer when the display reaches 0, and reset the countdown when a button is pressed.

Implementation Summary

A one-digit common-cathode 7-segment display, a reset pushbutton, and a buzzer were added to the Arduino Uno project. Segments A to F are mapped to PB0..PB5 (pins 8..13), segment G is mapped to PD3, and the common cathode is connected to GND through a single 1kOhm resistor so the hardware matches the assignment equipment list. The reset button is connected to PD2 (pin 2) and the buzzer to PD4 (pin 4). The countdown starts from 9, decrements once per second, and enters an alarm loop when 0 is reached.

Challenge and Hardware Mapping

The assignment states the 7-segment display should be connected to Port B and also lists one 1kOhm resistor in the required equipment. On the ATmega328P used in Arduino Uno, Port B has eight bits, but only PB0..PB5 are externally available on the board. Because a 7-segment digit requires seven segment signals, one practical adaptation was necessary: segments A..F are connected to PB0..PB5, while segment G is connected to PD3. To stay consistent with the single-resistor hardware list, the solution uses a common-cathode display with one shared 1kOhm resistor between the COM pin and GND. This preserves direct-register control and keeps most display lines on Port B while remaining compatible with the actual Arduino Uno hardware.

Signal	Microcontroller pin	Arduino pin
Segment A	PB0	8
Segment B	PB1	9
Segment C	PB2	10
Segment D	PB3	11
Segment E	PB4	12
Segment F	PB5	13
Segment G	PD3	3
Display COM	Through 1kOhm resistor	GND
Reset button	PD2	2
Buzzer	PD4	4

```
; Exercise 4
; Countdown on a 7-segment display with reset button and buzzer.

#define __SFR_OFFSET 0

#include "avr/io.h"

.global main

main:
  ldi   r18, 0x3F           ; Configure PB0..PB5 as outputs for segments A..F
  out   DDRB, r18           ; Enable the six Port B segment outputs
  sbi   DDRD, 3             ; Configure PD3 as the output for segment G
  sbi   DDRD, 4             ; Configure PD4 as the buzzer output
  cbi   DDRD, 2             ; Keep PD2 as the reset-button input
  sbi   PORTD, 2            ; Enable the internal pull-up on the button pin
  cbi   PORTD, 3            ; Start with segment G turned off
  cbi   PORTD, 4            ; Start with the buzzer turned off
  ldi   r16, 9              ; Load the initial digit value

show_digit:
  call  display_digit       ; Update the 7-segment display for the current digit
  cpi   r16, 0              ; Check whether the countdown reached zero
  breq  alarm_mode          ; Start the alarm when digit 0 is displayed
  ldi   r19, 100            ; Build a 1 second pause from 100 x 10ms slices

wait_second:
  ldi   r25, hi8(10)        ; Load a 10ms delay high byte
  ldi   r24, lo8(10)        ; Load a 10ms delay low byte
  call  delay_ms            ; Wait for one time slice
  sbis  PIND, 2             ; Skip next instruction if the button is not pressed
  rjmp  reset_counter       ; Reset immediately when the button is pressed
  dec   r19                 ; Count down one 10ms slice
  brne  wait_second         ; Stay here until 1 second elapsed
  dec   r16                 ; Move to the next digit in the countdown
  rjmp  show_digit          ; Refresh the display with the new digit

reset_counter:
  cbi   PORTD, 4            ; Silence the buzzer before restarting

wait_reset_release:
  sbis  PIND, 2             ; Skip next instruction after the button is released
  rjmp  wait_reset_release  ; Wait until the reset button is released
  ldi   r16, 9              ; Restore the start digit
  rjmp  show_digit          ; Start the countdown from the beginning

alarm_mode:
```

```
                            ; The display already shows 0 here, so only the buzzer
loop is needed
alarm_loop:
  sbis  PIND, 2             ; Skip next instruction while the button is released
  rjmp  reset_counter       ; Reset the countdown when the button is pressed
  sbi   PORTD, 4            ; Drive the buzzer output high
  ldi   r25, hi8(1)         ; Load the high byte for a 1ms half-period
  ldi   r24, lo8(1)         ; Load the low byte for a 1ms half-period
  call  delay_ms            ; Keep the buzzer high briefly
  cbi   PORTD, 4            ; Drive the buzzer output low
  ldi   r25, hi8(1)         ; Reload the high byte for the low half-period
  ldi   r24, lo8(1)         ; Reload the low byte for the low half-period
  call  delay_ms            ; Keep the buzzer low briefly
  rjmp  alarm_loop          ; Continue the alarm until reset

display_digit:
  cpi   r16, 0              ; Compare the current digit with 0
  breq  digit_0             ; Branch when the display should show 0
  cpi   r16, 1              ; Compare the current digit with 1
  breq  digit_1             ; Branch when the display should show 1
  cpi   r16, 2              ; Compare the current digit with 2
  breq  digit_2             ; Branch when the display should show 2
  cpi   r16, 3              ; Compare the current digit with 3
  breq  digit_3             ; Branch when the display should show 3
  cpi   r16, 4              ; Compare the current digit with 4
  breq  digit_4             ; Branch when the display should show 4
  cpi   r16, 5              ; Compare the current digit with 5
  breq  digit_5             ; Branch when the display should show 5
  cpi   r16, 6              ; Compare the current digit with 6
  breq  digit_6             ; Branch when the display should show 6
  cpi   r16, 7              ; Compare the current digit with 7
  breq  digit_7             ; Branch when the display should show 7
  cpi   r16, 8              ; Compare the current digit with 8
  breq  digit_8             ; Branch when the display should show 8
  ldi   r18, 0x2F           ; Load the Port B pattern for digit 9
  out   PORTB, r18          ; Enable segments A, B, C, D, and F
  sbi   PORTD, 3            ; Turn on segment G
  ret                       ; Return to the caller

digit_0:
  ldi   r18, 0x3F           ; Load the Port B pattern for digit 0
  out   PORTB, r18          ; Enable segments A..F
  cbi   PORTD, 3            ; Turn segment G off
  ret                       ; Return to the caller

digit_1:
  ldi   r18, 0x06           ; Load the Port B pattern for digit 1
```

```
  out   PORTB, r18          ; Enable segments B and C
  cbi   PORTD, 3            ; Turn segment G off
  ret                       ; Return to the caller

digit_2:
  ldi   r18, 0x1B           ; Load the Port B pattern for digit 2
  out   PORTB, r18          ; Enable segments A, B, D, and E
  sbi   PORTD, 3            ; Turn segment G on
  ret                       ; Return to the caller

digit_3:
  ldi   r18, 0x0F           ; Load the Port B pattern for digit 3
  out   PORTB, r18          ; Enable segments A, B, C, and D
  sbi   PORTD, 3            ; Turn segment G on
  ret                       ; Return to the caller

digit_4:
  ldi   r18, 0x26           ; Load the Port B pattern for digit 4
  out   PORTB, r18          ; Enable segments B, C, and F
  sbi   PORTD, 3            ; Turn segment G on
  ret                       ; Return to the caller

digit_5:
  ldi   r18, 0x2D           ; Load the Port B pattern for digit 5
  out   PORTB, r18          ; Enable segments A, C, D, and F
  sbi   PORTD, 3            ; Turn segment G on
  ret                       ; Return to the caller

digit_6:
  ldi   r18, 0x3D           ; Load the Port B pattern for digit 6
  out   PORTB, r18          ; Enable segments A, C, D, E, and F
  sbi   PORTD, 3            ; Turn segment G on
  ret                       ; Return to the caller

digit_7:
  ldi   r18, 0x07           ; Load the Port B pattern for digit 7
  out   PORTB, r18          ; Enable segments A, B, and C
  cbi   PORTD, 3            ; Turn segment G off
  ret                       ; Return to the caller

digit_8:
  ldi   r18, 0x3F           ; Load the Port B pattern for digit 8
  out   PORTB, r18          ; Enable segments A..F
  sbi   PORTD, 3            ; Turn segment G on
  ret                       ; Return to the caller

delay_ms:
```

```
                            ; Delay about (r25:r24) milliseconds at 16MHz
  ldi   r31, hi8(4000)      ; Load the inner-loop counter high byte
  ldi   r30, lo8(4000)      ; Load the inner-loop counter low byte
delay_inner:
  sbiw  r30, 1              ; Spend four clock cycles per inner-loop pass
  brne  delay_inner         ; Stay in the inner loop until it reaches zero
  sbiw  r24, 1              ; Decrement the millisecond counter
  brne  delay_ms            ; Repeat until the whole delay elapsed
  ret                       ; Return to the caller
```

Program Design and Logic

- The current digit is stored in register r16.
- The subroutine display_digit maps digit values 0..9 to the correct segment combination.
- The one-second pause is implemented as one hundred 10ms delay slices, which also makes it possible to poll the reset button during the countdown.
- When digit 0 is reached, the display remains at 0 and the program toggles the buzzer output continuously to generate sound.
- Pressing the reset button silences the buzzer, reloads digit 9, and starts the countdown again.

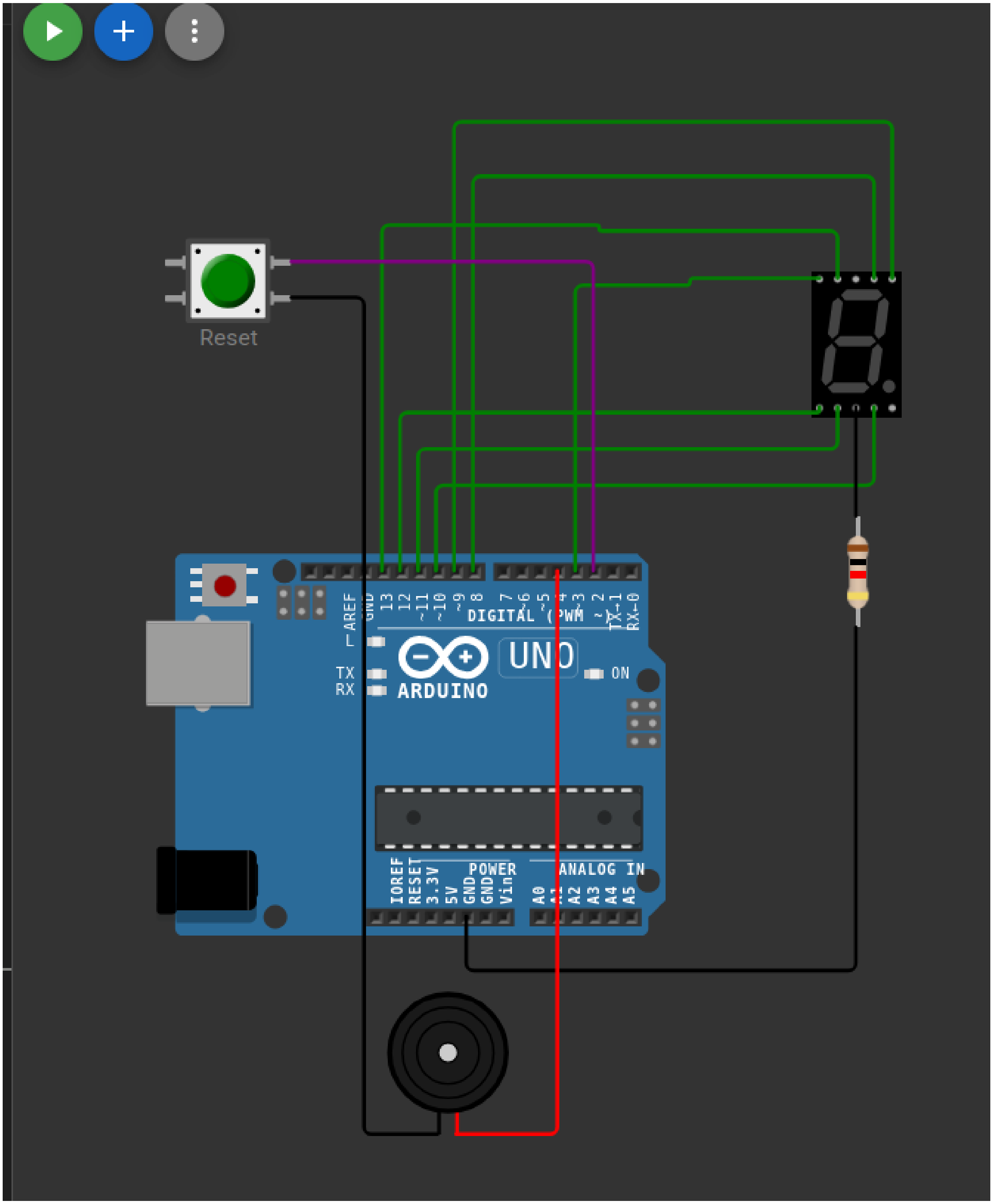

Fig. 6 Exercise 4

Challenges and Strategies

This exercise combines three I/O behaviors: static display control, one-second timing and asynchronous reset handling. A full interrupt-based solution was not required, so a polling strategy was used. The countdown loop checks the button every 10ms, which is fast enough to feel responsive while remaining easy to understand in assembly language. A shared resistor on the common cathode selected because the assignment specifies one resistor for the display setup, the software logic therefore keeps the common pin fixed at GND through that resistor and lights segments by driving their output pins HIGH.

Exercise-4.mp4

Video

5. CONCLUSION

Practical Work N2 demonstrates how direct AVR register control can be used to solve progressively more complex embedded tasks on the Arduino Uno platform. The work begins with a basic blinking LED, extends to running lights, then adds button driven state changes, and finally combines timing, display control, reset logic and sound generation in one program.

The exercises show the main strengths of AVR assembly language in educational tasks: each instruction has an observable hardware effect, timing can be controlled explicitly and the relationship between Arduino pins and ATmega328P ports becomes clear. At the same time, the work highlights practical constraints of the target hardware, such as the limited number of externally available Port B pins on the Arduino Uno.